

Testeo TechMind

Federico G. Gutierrez
Tester QA

VERSIÓN 1.1

CONTENIDOS

- 1** Informe de resultados
- 2** Detalle por categorías
- 3** Próximas tareas

Informe de resultados

Proyecto: TechMind - Organización Inteligente del Conocimiento Técnico

Responsable QA: Federico G. Gutierrez

Fecha de Ejecución: 22 de Julio de 2026

Resumen Ejecutivo

Durante el Sprint 1 se ejecutó la suite completa de pruebas sobre los endpoints de la API FastAPI y la integración con PostgreSQL. Se logró una cobertura total de escenarios funcionales, de límites, seguridad, resiliencia y validación de datos.

Métricas Generales

- Casos Planificados: 20
- Casos Ejecutados: 20
- Casos Exitosos: 20
- Casos Fallidos: --
- **Porcentaje de Éxito: 100 %**

Desglose por Tipo de Prueba

Categoría	Planificados	PASÓ	FALLÓ	% Éxito
Funcionales (Flujo Feliz)	7	7	--	100
Casos Borde / Encoding (UTF-8)	3	3	--	100
Validación de Esquema / Tipos	1	1	--	100
Seguridad (Inyección SQL / Content-Type)	1	1	--	100
Endpoints Complementarios	2	2	--	100
Integridad de Datos	6	6	--	100

Conclusión y Recomendaciones

El MVP cumple con todos los criterios de aceptación funcionales, de rendimiento (< 2000 ms) y de seguridad especificados para el Sprint 1. La API demuestra alta estabilidad y resiliencia ante errores de infraestructura.

Detalle

Testeo Funcionales (Flujo Feliz) — [7 casos]

- **CP-01:** Valida que la API clasifique correctamente un contenido técnico válido devuelto con HTTP 200.
- **CP-02:** Comprueba la extracción y retorno correcto de la lista de palabras clave relevantes en la respuesta.
- **CP-03:** Verifica la generación de un score de probabilidad de clasificación válido entre 0 y 1.
- **CP-09:** Confirma que la estructura del objeto JSON devuelto contenga todas las claves y tipos de datos especificados.
- **CP-10:** Valida la correcta inicialización y carga de los modelos serializados (.joblib) al arrancar el servicio.
- **CP-11:** Mide el tiempo de respuesta del endpoint asegurando que la inferencia se ejecute en menos de 2000 ms.
- **CP-20:** Verifica que el modelo normalice y procese adecuadamente textos ingresados con tildes y mayúsculas sostenidas.

Casos Borde / Encoding (UTF-8) — [2 casos]

- **CP-12:** Evalúa el comportamiento y la gestión de memoria de la API ante payloads de gran tamaño (+500k caracteres).
- **CP-13:** Garantiza la sanitización y el soporte de codificación UTF-8 procesando caracteres especiales, etiquetas y emojis sin errores.
- **CP-21:** Evalúa cómo reacciona la API ante entradas no técnicas o sin sentido.

Validación de Esquema / Tipos — [1 caso]

- **CP-14:** Confirma el rechazo inmediato (HTTP 422) por parte de Pydantic al enviar tipos de datos no válidos (números o booleanos).

Seguridad (Inyección SQL / Content-Type) — [2 casos]

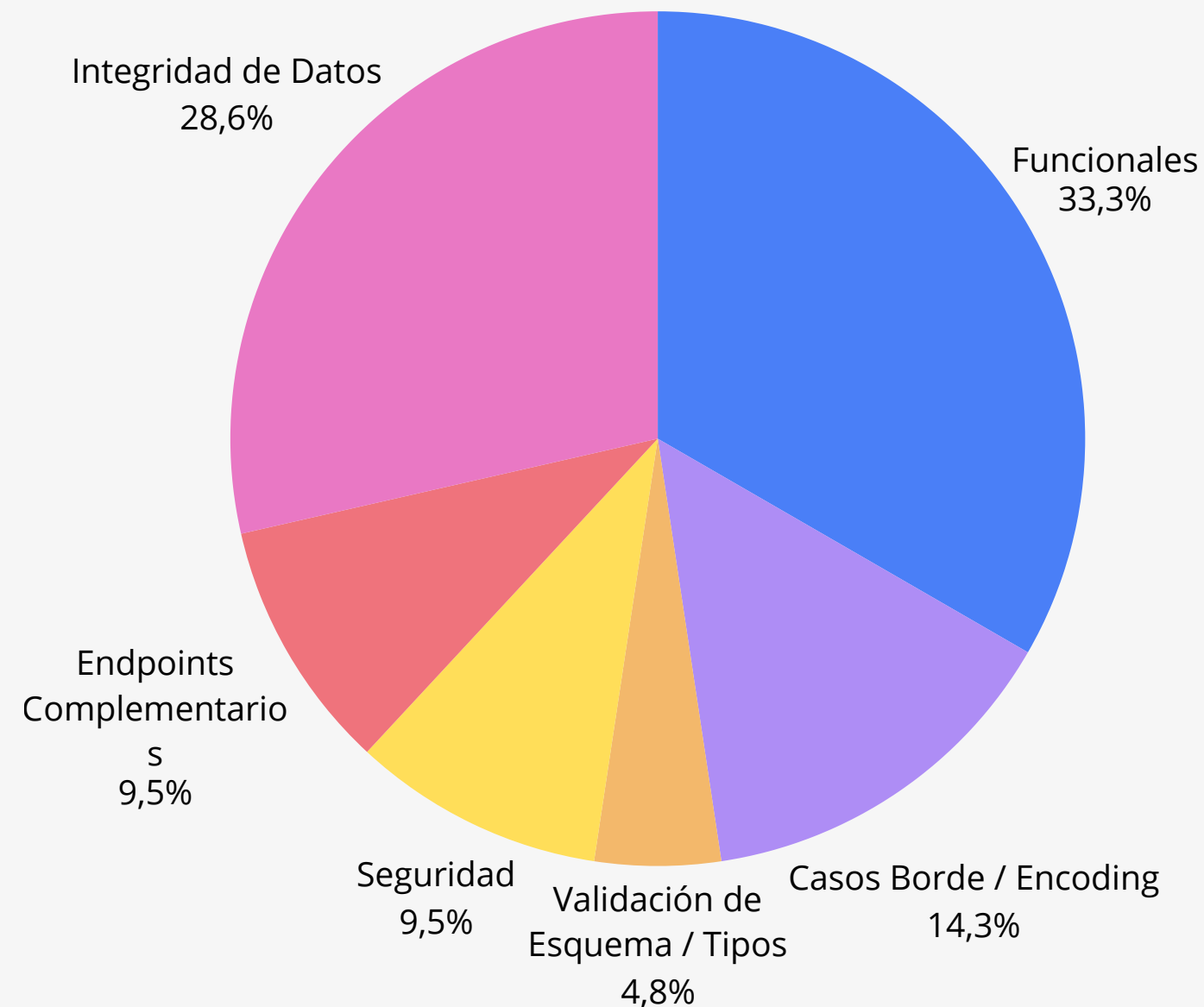
- **CP-15:** Comprueba la inmunidad ante intentos de inyección SQL guardando los comandos maliciosos como texto plano.
- **CP-16:** Valida el rechazo de solicitudes con formatos no soportados como XML mediante una respuesta HTTP 422.

Endpoints Complementarios — [2 casos]

- **CP-17:** Verifica la disponibilidad y estado operativo del microservicio mediante el endpoint de diagnóstico GET /health.
- **CP-18:** Valida que el endpoint GET /categorias retorne el catálogo completo con las 8 categorías del modelo.

Integridad de Datos — [6 casos]

- **CP-04:** Controla el rechazo de la solicitud (HTTP 422) cuando el campo "titulo" se envía vacío.
- **CP-05:** Controla el rechazo de la solicitud (HTTP 422) cuando el campo "texto" no contiene información.
- **CP-06:** Valida la respuesta de error adecuada al enviar tanto el título como el texto vacíos en el payload.
- **CP-07:** Comprueba la intercepción y manejo de excepciones ante la recepción de una estructura JSON mal formada.
- **CP-08:** Verifica el rechazo con código HTTP 405 Method Not Allowed al enviar una petición GET al endpoint POST /predecir.
- **CP-19:** Verifica que la API rechace la petición cuando se omiten claves obligatorias dentro del cuerpo JSON.



Matriz
Evidencias

Próximas tareas

Para continuar fortaleciendo la calidad y madurez del proyecto, se proponen las siguientes iniciativas para la próxima etapa:

- **Pruebas de Carga y Concurrencia (Stress Test):** Simular escenarios de alta demanda concurrente (+100 usuarios simultáneos) utilizando herramientas como Locust o JMeter, garantizando que la API y la base de datos PostgreSQL mantengan tiempos de respuesta óptimos bajo presión.
- **Automatización en Pipeline CI/CD (GitHub Actions + Newman):** Integrar la ejecución automática de la suite de pruebas de Postman en cada despliegue o pull request, asegurando la detección temprana de regresiones antes de pasar a entornos de producción.
- **Monitoreo Continuo y Alertas en Producción (OCI):** Configurar un esquema de lectura centralizada de logs en Oracle Cloud Infrastructure para alertar de forma automática ante caídas de servicio, errores 500 o fallos de conexión a la base de datos.